Learning DAX Through My Dashboard

AdventureWorks | Calculations, Measures, and Organisation

In this third journal, I document the DAX I created while following the Maven Analytics course and building my AdventureWorks report. I learned how reusable calculations connect business questions to KPI cards, charts, targets, customer details, and interactive scenarios.

Object type	Found in the PBIX	Purpose
Measures	44	Dynamic calculations and formatting helpers
Calculated columns	12	Stored row-level attributes in model tables
Calculated tables	3	Price scenario series and field parameters

The inventory and expressions in this PDF were extracted from the uploaded **AdventureWorks Report.pbix**. Formula whitespace is formatted for readability; object names and logic are preserved. This is a catalogue of the saved model, including learning exercises and helpers, not a claim that all objects are currently used by a visible report visual.

What I mean by calculated fields

“Calculated field” is a broad term in my learning notes. In this document I distinguish DAX calculated columns, measures, and calculated tables. Power Query custom columns such as the earlier Discount Price and Domain Name use **M**, so they are not included as DAX formulas here. [2]

How to use this journal

Start with organisation: see how Measure Table keeps calculations discoverable.
Read the catalogue: formulas are grouped by their purpose, with a short explanation after each.
Review the caveats: original learning formulas are preserved even when a more robust production version could be considered.

> **Scope:** the PBIX contains 42 measures in Measure Table, one in Customer Lookup, and one in Price Adjustment (%). I describe centralising measures as my organisational approach without claiming every measure has already been moved.

Organising Measures in a Measure Table

As the number of measures grew, I used a dedicated table called **Measure Table** to keep most of them together. This makes it easier to find calculations instead of searching across transaction and lookup tables.

How I create a dedicated measures table

1. Choose Home > Enter data and create a small table with one placeholder column.
2. Name it Measure Table and load it.
3. Select each existing measure and change its **Home table** to Measure Table; the control can appear under Measure tools or Properties depending on the Desktop version.
4. Hide the placeholder column, keeping the table visible.
5. Select Measure Table before creating further measures, or assign their Home table afterwards. [1]

I describe this as an “empty table” because it has no analytical data purpose. Technically, it can retain a hidden placeholder column and row. It does not need a relationship to the fact or dimension tables: a measure can reference other tables without its home table being related to them.

What the current PBIX contains

Home table	Measures	Example
Measure Table	42	Total Revenue, Return Rate, Adjusted Profit
Customer Lookup	1	Full Name (Customer Detail)
Price Adjustment (%)	1	Price Adjustment (%) Value

Moving a measure changes where it is organised, not its business definition or filter-context rules. Calculated columns remain in their appropriate tables; I do not move them into Measure Table.

A possible next organisational step

Display folders could group the 42 measures into Base KPIs, Returns, Time Intelligence, Targets, Customer Detail, Scenarios, and Formatting. In Model view, select a measure and set its Display folder property. These folder names are suggestions: the extracted PBIX currently has no display folders assigned. [1]

What I learned: naming and organisation help me maintain calculations, but they do not fix incorrect logic. I still need to understand each measure's grain, units, dependencies, and intended filter behaviour.

Core Measures: Quantities and Counts

Quantity Sold

Measure | Home table: Measure Table

```
Quantity Sold =
SUM('Sales Data'[OrderQuantity])
```

Sums units sold in the current filter context. Units are different from distinct orders.

Quantity Returned

Measure | Home table: Measure Table

```
Quantity Returned =
SUM(
    'Returns Data'[ReturnQuantity]
)
```

Sums returned units. This is the numerator used by Return Rate.

Average Retail Price

Measure | Home table: Measure Table

```
Average Retail Price =
AVERAGE(
    'Product Lookup'[ProductPrice]
)
```

Unweighted average of product catalogue prices in the current product context; not a quantity-weighted realised selling price.

Total Returns

Measure | Home table: Measure Table

```
Total Returns =
COUNT('Returns Data'[ReturnQuantity]
 )
```

Counts nonblank numeric ReturnQuantity records, not the total number of units returned. A record can contain more than one returned unit.

Total Orders

Measure | Home table: Measure Table

```
Total Orders =
DISTINCTCOUNT('Sales Data'[OrderNumber])
```

Counts distinct order numbers. An order can include several sales rows and product categories.

Total Customer

Measure | Home table: Measure Table

```
Total Customer =
DISTINCTCOUNT('Sales Data'[CustomerKey])
```

Counts distinct customer keys in Sales, so it represents purchasing customers in the current context. DISTINCTCOUNT can include a blank key if one is present.

Return Rate

Measure | Home table: Measure Table

```
Return Rate =
DIVIDE([Quantity Returned], [Quantity Sold], "No Sales")
```

Divides returned units by sold units. The original alternative result is the text "No Sales". Consider BLANK() for a purely numeric downstream measure; this PDF preserves the submitted formula.

Filtered Business Measures

Bulks Order

Measure | Home table: Measure Table

```
Bulks Order =
CALCULATE(
    [Total Orders],
    'Sales Data'[OrderQuantity]>1
)
```

Counts distinct orders with at least one sales line whose OrderQuantity exceeds one. It does not test whether the entire basket contains multiple units across separate lines.

Weekend Orders

Measure | Home table: Measure Table

```
Weekend Orders =
CALCULATE(
    [Total Orders],
    'Calendar Lookup'[Weekend] = "Weekend"
)
```

Restricts orders to dates labelled Weekend by the Calendar calculated column.

Bike Returns

Measure | Home table: Measure Table

```
Bike Returns =
CALCULATE(
    [Total Returns],
    'Product Categories Lookup'[CategoryName] = "Bikes"
)
```

Counts return records for the Bikes category, following the Total Returns definition. It does not sum returned units.

Bike Sales

Measure | Home table: Measure Table

```
Bike Sales =
CALCULATE(
    [Quantity Sold],
    'Product Categories Lookup'[CategoryName] = "Bikes"
)
```

Sums units sold for the Bikes category. Despite its name, this is not revenue or order count.

Bike Return Rate

Measure | Home table: Measure Table

```
Bike Return Rate =
CALCULATE(
    [Return Rate],
    'Product Categories Lookup'[CategoryName] = "Bikes"
)
```

Evaluates the unit-based Return Rate for Bikes through the product-category filter.

High Ticket Orders

Measure | Home table: Measure Table

```
High Ticket Orders =
CALCULATE(
    [Total Orders],
    FILTER(
        'Product Lookup',
        'Product Lookup'[ProductPrice] > [Overall Average Price]
    )
)
```

Filters products to those priced above Overall Average Price, then counts distinct orders containing qualifying products. FILTER supplies row-by-row product evaluation.

Filter Removal and Shares

All Orders

Measure | Home table: Measure Table

```
All Orders =
CALCULATE(
    [Total Orders],
    ALL(
        'Sales Data'
    )
)
```

Uses ALL on Sales Data within CALCULATE. Table-wide filter removal can affect related one-side dimensions through expanded-table semantics; test the intended denominator rather than assuming slicers remain active.

% of All Orders

Measure | Home table: Measure Table

```
% of All Orders =
DIVIDE(
    [Total Orders],
    [All Orders]
)
```

Current orders divided by All Orders. The denominator follows the filter-removal behaviour of that dependency.

Overall Average Price

Measure | Home table: Measure Table

```
Overall Average Price =
CALCULATE(
    [Average Retail Price],
    ALL('Product Lookup')
)
```

Removes filters from Product Lookup when computing the average catalogue price. This is a product-average benchmark, not transaction-weighted price.

All Returns

Measure | Home table: Measure Table

```
All Returns =
CALCULATE(
    [Total Returns],
    ALL(
        'Returns Data'
    )
)
```

Uses ALL on Returns Data to compute a broad return-record denominator. As with All Orders, inspect table-wide filter removal in the model.

% of All Returns

Measure | Home table: Measure Table

```
% of All Returns =
DIVIDE(
    [Total Returns],
    [All Returns]
)
```

Current return-record count divided by All Returns; not a share of returned units.

Revenue, Cost, and Profit

Total Revenue

Measure | Home table: Measure Table

```
Total Revenue =
SUMX(
    'Sales Data',
    'Sales Data'[OrderQuantity] *
    RELATED(
        'Product Lookup'[ProductPrice]
    )
)
```

Iterates sales rows with SUMX, multiplying quantity by the related product catalogue price. It does not incorporate transaction-specific discounts unless separately modelled.

Total Costs

Measure | Home table: Measure Table

```
Total Costs =
SUMX('Sales Data',
    'Sales Data'[OrderQuantity]
    *
    RELATED(
        'Product Lookup'[ProductCost]
    )
)
```

Iterates sales rows and multiplies quantity by the related product cost.

Total Profit

Measure | Home table: Measure Table

```
Total Profit =
[Total Revenue] - [Total Costs]
```

Revenue minus product costs. This is not net income after operating expenses, tax, or return adjustments.

Average Revenue per Customer

Measure | Home table: Measure Table

```
Average Revenue per Customer =
DIVIDE(
    [Total Revenue],
    [Total Customer]
)
```

Revenue divided by distinct purchasing customers. It is not revenue per order.

Time Intelligence

YTD Revenue

Measure | Home table: Measure Table

```
YTD Revenue =
CALCULATE(
    [Total Revenue],
    DATESYTD(
        'Calendar Lookup'[Date]
    )
)
```

Evaluates revenue for dates from the beginning of the calendar year to the current date context. Check date-table configuration.

Previous Month Revenue

Measure | Home table: Measure Table

```
Previous Month Revenue =
CALCULATE(
    [Total Revenue],
    DATEADD('Calendar Lookup'[Date],
        -1,
        MONTH
    )
)
```

Shifts the selected date context back one month. The result depends on the dates selected, not merely the latest calendar month in the file.

Previous Month Orders

Measure | Home table: Measure Table

```
Previous Month Orders =
CALCULATE(
    [Total Orders],
    DATEADD(
        'Calendar Lookup'[Date],
        -1,
        MONTH
    )
)
```

Evaluates distinct orders with the date context shifted back one month.

Previous Month Returns

Measure | Home table: Measure Table

```
Previous Month Returns =
CALCULATE(
    [Total Returns],
    DATEADD(
        'Calendar Lookup'[Date],
        -1,
        MONTH
    )
)
```

Evaluates return-record count with dates shifted back one month. It does not sum returned units.

Previous Month Profit

Measure | Home table: Measure Table

```
Previous Month Profit =
CALCULATE(
    [Total Profit],
    DATEADD(
        'Calendar Lookup'[Date],
        -1,
        MONTH
    )
)
```

Evaluates product profit with dates shifted back one month.

10-Day Rolling Revenue

Measure | Home table: Measure Table

```
10-Day Rolling Revenue =
CALCULATE(
    [Total Revenue],
    DATESINPERIOD('Calendar Lookup'[Date],
        MAX(
            'Calendar Lookup'[Date]),
        -10,
        DAY
    )
)
```

Uses the maximum date in context as the anchor and applies a backward ten-day date window.

90-Day Rolling Profit

Measure | Home table: Measure Table

```
90-Day Rolling Profit =
CALCULATE(
    [Total Profit],
    DATESINPERIOD('Calendar Lookup'[Date],
        MAX(
            'Calendar Lookup'[Date]),
        -90,
        DAY
    )
)
```

Applies a backward 90-day date window ending at the maximum date in context.

Targets and Target Gaps

Revenue Target

Measure | Home table: Measure Table

```
Revenue Target =
[Previous Month Revenue] * 1.1
```

Sets the target to 110% of previous-month revenue. It is a project rule, not a documented stakeholder-approved forecast.

Order Target

Measure | Home table: Measure Table

```
Order Target =
[Previous Month Orders] * 1.1
```

Sets the order target to 110% of previous-month orders; the calculated target can be fractional even though actual orders are whole-number counts.

Profit Target

Measure | Home table: Measure Table

```
Profit Target =
[Previous Month Profit] * 1.1
```

Sets the profit target to 110% of previous-month product profit.

Order Target Gap

Measure | Home table: Measure Table

```
Order Target Gap =
[Total Orders] - [Order Target]
```

Actual orders minus the order target. Positive means above the target; negative means below.

Revenue Target Gap

Measure | Home table: Measure Table

```
Revenue Target Gap =
[Total Revenue] - [Revenue Target]
```

Actual revenue minus the revenue target.

Profit Target Gap

Measure | Home table: Measure Table

```
Profit Target Gap =
[Total Profit] - [Profit Target]
```

Actual product profit minus the profit target.

Customer Detail Helpers

Total Revenue (Customer Detail)

Measure | Home table: Measure Table

```
Total Revenue (Customer Detail) =
IF(
    HASONEVALUE(
        'Customer Lookup'[CustomerKey]
    ),
    [Total Revenue],
    "-"
)
```

Shows revenue when exactly one customer key is in context; otherwise returns the text "-". This mixes numeric and text outputs.

Total Orders (Customer Detail)

Measure | Home table: Measure Table

```
Total Orders (Customer Detail) =
IF(
    HASONEVALUE(
        'Customer Lookup'[CustomerKey]
    ),
    [Total Orders],
    "-"
)
```

Shows distinct orders for a single customer context; otherwise returns "-".

Full Name (Customer Detail)

Measure | Home table: Customer Lookup

```
Full Name (Customer Detail) =
IF(
    HASONEVALUE(
        'Customer Lookup'[CustomerKey]
    ),
    MAX(
        'Customer Lookup'[Full Name]
    ),
    "Multiple Customers"
 )
```

A measure, not the Full Name calculated column. Shows the name for a single customer context, otherwise "Multiple Customers". Its home table is Customer Lookup.

Scenario Measures

Price Adjustment (%) Value

Measure | Home table: Price Adjustment (%)

```
Price Adjustment (%) Value =
SELECTEDVALUE('Price Adjustment (%)'[Price Adjustment (%)], 0)
```

Reads the selected numeric scenario value, defaulting to zero if no single value is available. Its home table is the parameter table.

Adjusted Price

Measure | Home table: Measure Table

```
Adjusted Price =
[Average Retail Price] * (1 + 'Price Adjustment (%)'[Price Adjustment (%) Value])
```

Multiplies Average Retail Price by one plus the selected adjustment. A value of 0.1 means +10%, not +0.1%.

Adjusted Revenue

Measure | Home table: Measure Table

```
Adjusted Revenue =
SUMX(
    'Sales Data',
    'Sales Data'[OrderQuantity] *
    [Adjusted Price]
)
```

Iterates sales rows and multiplies quantity by the Adjusted Price measure. Calling a measure inside SUMX invokes context transition; its result depends on the row and model relationships. Validate single-product and aggregated results.

Adjusted Profit

Measure | Home table: Measure Table

```
Adjusted Profit =
[Adjusted Revenue] - [Total Costs]
```

Adjusted Revenue minus unchanged Total Costs. This scenario does not model demand response or changing costs.

Dynamic Formatting

Metric Line Color

Measure | Home table: Measure Table

```
Metric Line Color =
VAR MetricID =
    SELECTEDVALUE(
        'Product Metric Selection'[Product Metric Selection Order],
        1
    )
RETURN
    SWITCH(
        TRUE(),
        MetricID IN {0, 1, 2}, "#333333",
        MetricID IN {3, 4}, "#FF0000",
        "#333333"
    )
```

Uses the product field-parameter ordinal to choose a colour: dark for Orders/Revenue/Profit and red for Returns/Return %. VAR stores the selected ordinal; SWITCH determines the result.

Customer Metric Line Color

Measure | Home table: Measure Table

```
Customer Metric Line Color =
VAR MetricID =
    SELECTEDVALUE(
        'Customer Metric Selection'[Customer Metric Selection Order],
        0
    )
RETURN
    SWITCH(
        MetricID,
        0, "#333333",
        1, "#20E2D7",
        "#333333"
    )
```

Uses the customer field-parameter ordinal to choose dark for customer count and teal for revenue per customer.

Calculated Columns: Row-Level Attributes

These twelve expressions are stored model columns, evaluated row by row at refresh in an Import model. They support labels, classifications, sorting, and measure filters. They remain in their business tables. [2]

Quantity Type

Calculated column | Table: Sales Data

```
Quantity Type =
IF(
    'Sales Data'[OrderQuantity]>1,
    "Multiple Items",
    "Single Item"
)
```

Classifies a sales line as Multiple Items when quantity exceeds one; otherwise Single Item. This is a line-level label, not a complete order-basket classification.

Is Parent?

Calculated column | Table: Customer Lookup

```
Is Parent? =
IF('Customer Lookup'[TotalChildren] >0, "Yes", "No")
```

Labels customers with TotalChildren greater than zero as Yes, otherwise No. Missing values need separate interpretation if present.

Full Name

Calculated column | Table: Customer Lookup

```
Full Name =
'Customer Lookup'[Prefix] & " " & 'Customer Lookup'[FirstName] & " " & 'Customer
Lookup'[LastName]
```

Concatenates prefix, first name, and last name. This is a calculated column; Full Name (Customer Detail) is a separate measure.

Month Number (DAX)

Calculated column | Table: Calendar Lookup

```
Month Number (DAX) =
SWITCH(
    'Calendar Lookup'[Month name],
    "January", "1",
    "February", "2",
    "March","3",
    "April","4",
    "May","5",
    "June","6",
    "July","7",
    "August","8",
    "September","9",
    "October","10",
    "November","11",
    "December","12",
    "Other"
)
```

Maps English month names to quoted numbers. Because the outputs are strings, this is a text result; numeric month sorting should use a numeric column or a revised expression with numeric results.

Price Point

Calculated column | Table: Product Lookup

```
Price Point =
SWITCH(
    TRUE(),
    'Product Lookup'[ProductPrice] > 500, "High",
    'Product Lookup'[ProductPrice] >100, "Mid-Range",
    "Low"
)
```

Classifies prices above 500 as High, above 100 as Mid-Range, and all others as Low. Order matters because SWITCH stops at the first match.

Income Level

Calculated column | Table: Customer Lookup

```
Income Level =
SWITCH(
    TRUE(),
    'Customer Lookup'[AnnualIncome] >= 150000, "Very High",
    'Customer Lookup'[AnnualIncome] >= 100000, "High",
    'Customer Lookup'[AnnualIncome] >= 50000, "Average",
    "Low"
)
```

Classifies income into Very High, High, Average, or Low using descending thresholds. These are project-defined bands, not externally validated customer segments.

Customer Priority

Calculated column | Table: Customer Lookup

```
Customer Priority =
IF(
    'Customer Lookup'[AnnualIncome] > 100000 &&
    'Customer Lookup'[Is Parent?] = "Yes",
    "Priority",
    "Standard"
)
```

Combines income above 100,000 with Is Parent? = Yes using &&. The rule is a learning classification rather than a demonstrated targeting strategy.

Education Category

Calculated column | Table: Customer Lookup

```
Education Category =
SWITCH(
    'Customer Lookup'[EducationLevel],
    "High School", "High School",
    "Partial High School", "High School",
    "Bachelors", "Undergrad",
    "Partial College", "Undergrad",
    "Graduate Degree", "Graduate"
)
```

Maps education labels into High School, Undergrad, or Graduate. Without an explicit fallback, unmatched labels return BLANK().

Month Short

Calculated column | Table: Calendar Lookup

```
Month Short =
UPPER
    (LEFT(
    'Calendar Lookup'[Month name],3)
    )
```

Takes the first three characters of Month name and converts them to uppercase.

Day of Week

Calculated column | Table: Calendar Lookup

```
Day of Week =
WEEKDAY('Calendar Lookup'[Date],2)
```

WEEKDAY with return_type 2 numbers Monday as 1 through Sunday as 7.

Weekend

Calculated column | Table: Calendar Lookup

```
Weekend =
IF(
    'Calendar Lookup'[Day of Week] IN {6,7},
    "Weekend",
    "Weekday"
)
```

Uses membership in {6,7} to label Saturday and Sunday as Weekend, otherwise Weekday.

Birth Year

Calculated column | Table: Customer Lookup

```
Birth Year =
YEAR('Customer Lookup'[BirthDate])
```

Extracts the year from BirthDate. It is a birth year, not current age.

Calculated Tables and Parameters

These tables also use DAX, but their expressions return tables rather than single values. The parameter UI can generate them, so not every line needs to be typed manually. Their definitions belong in a complete model inventory.

Price Adjustment (%)

Calculated table | Table: Price Adjustment (%)

```
Price Adjustment (%) =
GENERATESERIES(-1, 1, 0.1)
```

Creates a range from -1 to +1 in steps of 0.1: -100% to +100% in ten-percentage-point increments. A separate selected-value measure reads the current input.

Product Metric Selection

Calculated table | Table: Product Metric Selection

```
Product Metric Selection =
{
    ("Orders", NAMEOF('Measure Table'[Total Orders]), 0),
    ("Revenue", NAMEOF('Measure Table'[Total Revenue]), 1),
    ("Profit", NAMEOF('Measure Table'[Total Profit]), 2),
    ("Returns", NAMEOF('Measure Table'[Total Returns]), 3),
    ("Return %", NAMEOF('Measure Table'[Return Rate]), 4)
}
```

A field parameter containing five labelled measure references and their order. NAMEOF preserves object references used by the parameter; the table expression alone is not the complete field-parameter metadata configuration.

Customer Metric Selection

Calculated table | Table: Customer Metric Selection

```
Customer Metric Selection =
{
    ("Total Customer", NAMEOF('Measure Table'[Total Customer]), 0),
    ("Revenue per Customer", NAMEOF('Measure Table'[Average Revenue per Customer]), 1)
}
```

A field parameter switching between purchasing-customer count and revenue per customer. The ordinal values are also read by the customer colour measure.

DAX Functions I Practised

Category	Functions present in the saved formulas
Aggregation and counts	SUM, COUNT, DISTINCTCOUNT, AVERAGE, MAX
Iteration and relationships	SUMX, RELATED
Filters and context	CALCULATE, FILTER, ALL
Ratios	DIVIDE
Logic and selection	IF, SWITCH, TRUE, HASONEVALUE, SELECTEDVALUE
Dates and time	YEAR, WEEKDAY, DATESYTD, DATEADD, DATESINPERIOD
Text	LEFT, UPPER
Table / parameter definitions	GENERATESERIES, NAMEOF

Other syntax includes VAR / RETURN, IN, &&, &, arithmetic operators, measure references in square brackets, and table constructors using braces. These are language elements, not all separate DAX functions.

What these calculations connect to

Executive page: totals, monthly comparisons, category measures, and return metrics.
Map: Total Orders filtered through geography.
Product Detail: targets, gaps, selected metric, price scenario, and line colours.
Customer Detail: distinct customers, revenue per customer, single-customer helpers, and metric selection.
Learning exercises: additional filtered and rolling measures may exist without being bound to a current visual.

Checks I would carry into a production report

Keep units and record counts separate. Check denominator filter removal. Treat text alternatives in numeric measures carefully. Validate date windows and target rules. Test the price scenario across products and totals without claiming a demand forecast. Use numeric month values for sorting. Organise measures without changing their meaning.

My main lesson: learning DAX is not just learning function names. I need to understand what is counted, which filters apply, how dependencies are evaluated, and whether the result answers the business question.

Evidence and references

Primary project evidence: expressions extracted from the uploaded AdventureWorks Report.pbix; the supplied DAX screenshots provide additional examples. Extraction identifies saved objects, not a full correctness or visual-usage audit. All 44 measures, 12 calculated columns, and 3 calculated-table expressions are included above.

[1] Microsoft Learn: Create measures and organise measures
[2] Microsoft Learn: Calculation options in Power BI Desktop

Author: iffaaulia | Guided learning materials: Maven Analytics | Documentation prepared 26 October 2026.